

Jenkins-01-Tasks.

1. Install Jenkins and run Jenkins on port number 8081.

Task 1 — Install Jenkins and Run Jenkins on Port 8081

Objective

Install Jenkins on an **Amazon Linux 2023 EC2 instance**, configure Jenkins to use **port 8081 instead of 8080**, and access Jenkins from the browser.

1. Launch EC2 Instance

AWS Console:

EC2

→ Instances

→ Launch instance

Configuration used:

Name: Jenkins-Server

AMI: Amazon Linux 2023

Instance type: t3.micro

Key pair: my_server

Storage: 20 GiB gp3

Auto-assign Public IP: Enabled

Security Group Rules

Add:

Type: SSH

Protocol: TCP

Port: 22

Source: My IP

Add:

Type: Custom TCP

Protocol: TCP

Port: 8081

Source: My IP

Port 22 is for SSH.

Port 8081 is for accessing Jenkins.

2. Connect to EC2

On local machine:

```
chmod 400 my_server.pem
```

Connect:

```
ssh -i "my_server.pem" ec2-user@<EC2-PUBLIC-DNS>
```

Example prompt after successful connection:

```
[ec2-user@ip-172-31-x-x ~]$
```

Become root:

```
sudo su -
```

3. Install Java 21

First check:

```
java -version
```

Initially we received:

```
java: command not found
```

Install Amazon Corretto Java 21:

```
dnf install java-21-amazon-corretto-devel -y
```

Verify:

```
java -version
```

Expected output:

```
openjdk version "21..."
OpenJDK Runtime Environment Corretto-21...
```

In our server:

```
openjdk version "21.0.12.1"
```

So Java installation was successful.

4. Add Jenkins Repository

Download Jenkins stable repository:

```
wget -O /etc/yum.repos.d/jenkins.repo \
https://pkg.jenkins.io/rpm-stable/jenkins.repo
```

Import Jenkins signing key:

```
rpm --import https://pkg.jenkins.io/rpm-stable/jenkins.io-2026.key
```

Update packages:

```
dnf upgrade -y
```

5. Install Jenkins

Run:

```
dnf install jenkins -y
```

We successfully installed:

```
jenkins-2.568.3-1.noarch
```

At this point Jenkins was installed but we did **not start it yet**, because the requirement was to run Jenkins on port 8081.

6. Change Jenkins Port from 8080 to 8081

Run:

```
systemctl edit jenkins
```

The nano editor opened.

At the top of the file, **above**:

```
### Lines below this comment will be discarded
```

we added:

```
[Service]
Environment="JENKINS_PORT=8081"
```

It should look like:

```
### Editing /etc/systemd/system/jenkins.service.d/override.conf
### Anything between here and the comment below will become the new contents of the
file
```

```
[Service]
Environment="JENKINS_PORT=8081"
```

```
### Lines below this comment will be discarded
```

Save in nano

Ctrl + O

Enter

Ctrl + X

Important:

Ctrl + O = Save

Ctrl + X = Exit

7. Reload systemd

After changing the Jenkins configuration:

```
systemctl daemon-reload
```

8. Enable Jenkins

Run:

```
systemctl enable jenkins
```

This makes Jenkins start automatically when the EC2 server boots.

9. Start Jenkins

```
systemctl start jenkins
```

Check:

```
systemctl status jenkins
```

Expected:

Active: active (running)

To exit the status screen:

```
q
```

10. Verify Jenkins Port

Run:

```
ss -tulnp | grep 8081
```

Our output was:

```
tcp LISTEN 0 50 *:8081 *:~ users:(("java",pid=27472,fd=9))
```

This confirms:

Jenkins process → Running

Port → 8081

Protocol → TCP

11. Open Jenkins in Browser

Find the EC2 Public IPv4 address.

Open:

```
http://<EC2-PUBLIC-IP>:8081
```

Example:

```
http://32.196.117.219:8081
```

The browser displayed:

Unlock Jenkins

12. Get Jenkins Initial Admin Password

On the EC2 server:

```
cat /var/lib/jenkins/secrets/initialAdminPassword
```

Copy the generated password.

Paste it into:

Administrator password

Click:

Continue

13. Install Jenkins Plugins

On the **Customize Jenkins** page choose:

Install suggested plugins

Jenkins installs commonly required plugins automatically.

Wait for installation to complete.

14. Create Jenkins Admin User

Create an administrator account.

Example:

Username: chandrashekar

Password: *****

Full Name: Chandrashekar

Email: your-email

Click:

Save and Continue

Then:

Save and Finish

Finally:

Start using Jenkins

Final Architecture

Local Windows Machine

|
| SSH :22
|
v

AWS EC2

Amazon Linux 2023

|
|-- Java 21
|
|-- Jenkins
|
|-- Jenkins Port :8081
|
v

Browser

http://EC2-PUBLIC-IP:8081

Important Commands

java -version

systemctl start jenkins

systemctl stop jenkins

systemctl restart jenkins

systemctl status jenkins

systemctl enable jenkins

ss -tulnp | grep 8081

cat /var/lib/jenkins/secrets/initialAdminPassword

journalctl -u jenkins

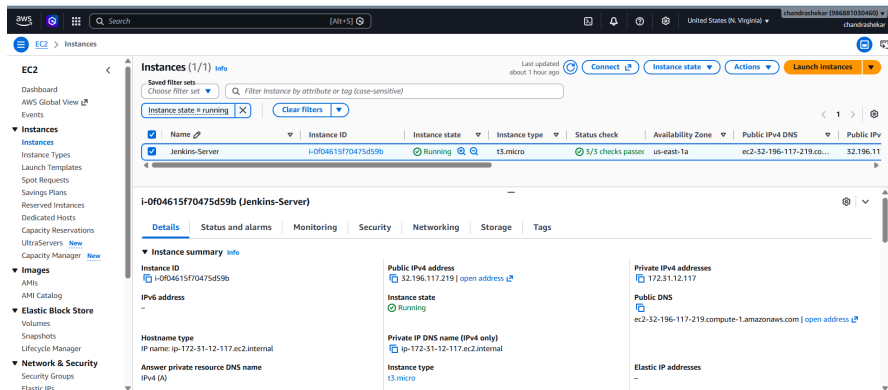
For live Jenkins logs:

journalctl -u jenkins -f

Task 1 Result

- ✓ EC2 instance created
- ✓ Amazon Linux 2023 configured
- ✓ Java 21 installed
- ✓ Jenkins repository configured
- ✓ Jenkins installed
- ✓ Jenkins port changed from 8080 → 8081
- ✓ Jenkins service started
- ✓ Port 8081 verified
- ✓ Jenkins opened in browser
- ✓ Jenkins unlocked
- ✓ Suggested plugins installed
- ✓ Jenkins administrator account created

Task 1 — Completed successfully.



```

root@ip-172-31-12-117:~#
Installing      : langpacks-core-font-en-3.0-21.amzn2023.0.4.noarch
Installing      : cairo-1.18.0-4.amzn2023.0.3.x86_64
Installing      : harfbuzz-2.0.0-2.amzn2023.0.2.x86_64
Installing      : freetype-2.13.2-5.amzn2023.0.2.x86_64
Installing      : fontconfig-2.13.94-2.amzn2023.0.2.x86_64
Running scriptlet: fontconfig-2.13.94-2.amzn2023.0.2.x86_64
Installing      : alsa-lib-1.2.7.2-1.amzn2023.0.4.x86_64
Installing      : java-21-amazon-corretto-headless-1:21.0.12-9-1.amzn2023.1.x86_64
Running scriptlet: java-21-amazon-corretto-headless-1:21.0.12-9-1.amzn2023.1.x86_64
Installing      : java-21-amazon-corretto-devel-1:21.0.12-9-1.amzn2023.1.x86_64
Running scriptlet: java-21-amazon-corretto-devel-1:21.0.12-9-1.amzn2023.1.x86_64
Running scriptlet: fontconfig-2.13.94-2.amzn2023.0.2.x86_64
Running scriptlet: java-21-amazon-corretto-devel-1:21.0.12-9-1.amzn2023.1.x86_64
Verifying      : alsa-lib-1.2.7.2-1.amzn2023.0.4.x86_64
Verifying      : cairo-1.18.0-4.amzn2023.0.3.x86_64
Verifying      : dejavu-sans-fonts-2.37-16.amzn2023.0.2.noarch
Verifying      : dejavu-sans-mono-fonts-2.37-16.amzn2023.0.2.noarch
Verifying      : dejavu-serif-fonts-2.37-16.amzn2023.0.2.noarch
Verifying      : fontconfig-2.13.94-2.amzn2023.0.2.x86_64
Verifying      : fonts-filesystem-1:2.0.5-12.amzn2023.0.2.noarch
Verifying      : freetype-2.13.2-5.amzn2023.0.2.x86_64
Verifying      : google-noto-fonts-common-20240401-1.amzn2023.0.2.noarch
Verifying      : google-noto-sans-vf-fonts-20240401-1.amzn2023.0.2.noarch
Verifying      : graphite2-1.3.14-7.amzn2023.0.3.x86_64
Verifying      : harfbuzz-2.0.0-2.amzn2023.0.2.x86_64
Verifying      : java-21-amazon-corretto-devel-1:21.0.12-9-1.amzn2023.1.x86_64
Verifying      : java-21-amazon-corretto-headless-1:21.0.12-9-1.amzn2023.1.x86_64
Verifying      : jpackage-filesystem-6.0.0-7.amzn2023.0.6.noarch
Verifying      : langpacks-core-font-en-3.0-21.amzn2023.0.4.noarch
Verifying      : libx11-1.8.10-2.amzn2023.0.1.x86_64
Verifying      : libx11-common-1.8.10-2.amzn2023.0.1.noarch
Verifying      : libxau-1.0.11-6.amzn2023.0.1.x86_64
Verifying      : libXext-1.3.6-1.amzn2023.0.1.x86_64
Verifying      : libXrender-0.9.11-6.amzn2023.0.1.x86_64
Verifying      : libXt-1.3.6-1.amzn2023.0.1.x86_64
Verifying      : libXtst-1.3.6-1.amzn2023.0.1.x86_64
Verifying      : libXv-1.0.9-4.amzn2023.0.1.x86_64
Verifying      : libXxx-1.17.0-1.amzn2023.0.1.x86_64
Verifying      : pixman-0.43.4-1.amzn2023.0.4.x86_64
Verifying      : x11-common-0.6.3-56.amzn2023.0.2.noarch

Installed:
  alsa-lib-1.2.7.2-1.amzn2023.0.4.x86_64      cairo-1.18.0-4.amzn2023.0.3.x86_64      dejavu-sa
  dejavu-sans-mono-fonts-2.37-16.amzn2023.0.2.noarch      dejavu-serif-fonts-2.37-16.amzn2023.0.2.noarch      fontconfi
  fonts-filesystem-1:2.0.5-12.amzn2023.0.2.noarch      freetype-2.13.2-5.amzn2023.0.2.x86_64      google-noto
  java-21-amazon-corretto-devel-1:21.0.12-9-1.amzn2023.1.x86_64      java-21-amazon-corretto-headless-1:21.0.12-9-1.amzn2023.1.x86_64      harfbuzz
  langpacks-core-font-en-3.0-21.amzn2023.0.4.noarch      jpackage-filesystem-6.0.0-7.amzn2023.0.6.noarch      java-packa
  libx11-1.8.10-2.amzn2023.0.1.x86_64      libx11-common-1.8.10-2.amzn2023.0.1.noarch      libxau-1.0.11-6
  libXext-1.3.6-1.amzn2023.0.1.x86_64      libXrender-0.9.11-6.amzn2023.0.1.x86_64      libXt-1.3.6-1
  libXtst-1.3.6-1.amzn2023.0.1.x86_64      libXv-1.0.9-4.amzn2023.0.1.x86_64      libXxx-1.17.0-1
  pixman-0.43.4-1.amzn2023.0.4.x86_64      x11-common-0.6.3-56.amzn2023.0.2.noarch      x11-comm

Complete!
root@ip-172-31-12-117:~# java -version
openjdk version "21.0.12" 2024-08-18 LTS
OpenJDK Runtime Environment Corretto-21.0.12.9.1 (build 21.0.12.1-9-LTS)
openjdk 64-bit Server VM Corretto-21.0.12.9.1 (build 21.0.12.1-9-LTS, mixed mode, sharing)
root@ip-172-31-12-117:~#

```

```

root@ip-172-31-12-117:~#
/etc/systemd/system/jenkins.service.d/ #override.conf1161823248008de Modified
-- GNU nano 3.1 --
# vim:ft=conf:et:sw=4:sts=4:ts=8
# Editing /etc/systemd/system/jenkins.service.d/override.conf
# Anything between here and the comment below will become the new contents of the file

[Service]
Environment="JENKINS_PORT=8081"

## Lines below this comment will be discarded

## /usr/lib/systemd/system/jenkins.service
# This file is managed by systemd(). Do NOT edit this file manually!
# To override these settings, run:
#   systemctl edit jenkins
# For more information about drop-in files, see:
#   https://www.freedesktop.org/software/systemd/man/systemd.unit.html

[Unit]
Description=Jenkins Continuous Integration Server
Requires=network.target
After=network.target
StartsAfter=systemd-journald.service

[Service]
Type=simple
ExecStart=/usr/bin/java -jar /usr/share/java/jenkins.war
Restart=on-failure
SuccessExitStatus=143

# Configures the time to wait for start-up. If Jenkins does not signal start-up
# completion within the configured time, the service will be considered failed
# and will be shut down again. Takes a unit-less value in seconds, or a time span
# value such as "5min 20s". Pass "infinity" to disable the timeout logic.
TimeoutStartSec=90

# Only account that runs the Jenkins daemon
# Be careful when you change this, as you need to update the permissions of
# $JENKINS_HOME, $JENKINS_LOG, and (if you have already run Jenkins)
# $JENKINS_WAR.
User=jenkins
Group=jenkins

# Directory where Jenkins stores its configuration and workspaces
Environment="JENKINS_HOME=/usr/lib/jenkins"
WorkingDirectory=/usr/lib/jenkins

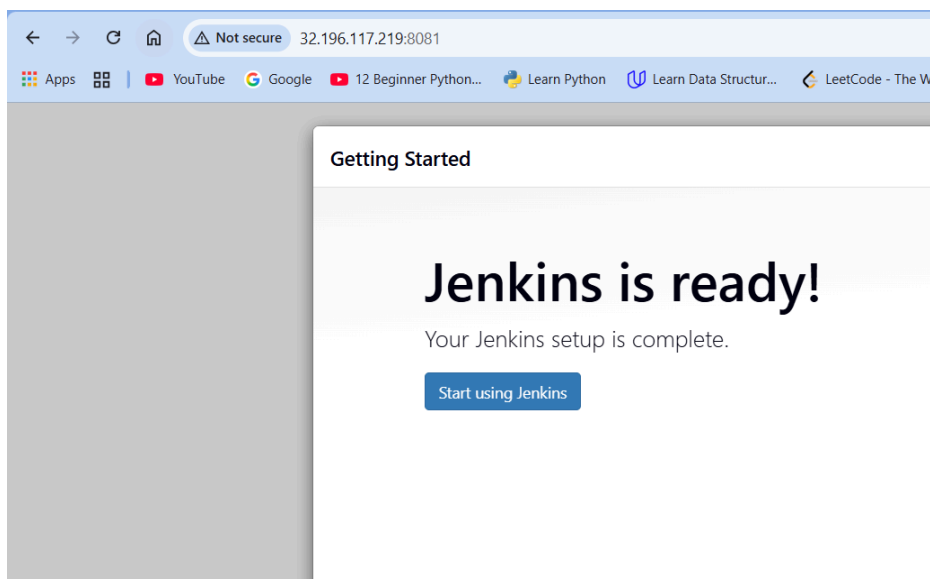
# Location of the Jenkins WAR
Environment="JENKINS_WAR=/usr/share/java/jenkins.war"

help write out where is Cat Execute Location Undo Set Mark To Bracket Prev/next Back Forward Prev word Next word
exit read file replace paste

```

```
Transaction Summary
Install 1 Package
Total download size: 96 M
Installed size: 96 M
Downloading Packages:
jenkins-2.568.3-1.noarch.rpm
Total
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
Preparing
Running scriptlet: jenkins-2.568.3-1.noarch
Installing
Running scriptlet: jenkins-2.568.3-1.noarch
Verifying
Running transaction
Installed:
jenkins-2.568.3-1.noarch
Complete!
[root@ip-172-31-12-117 ~]# systemctl edit jenkins
[root@ip-172-31-12-117 ~]# systemctl daemon-reload
[root@ip-172-31-12-117 ~]# systemctl start jenkins
systemctl status jenkins
Created symlink /etc/systemd/system/multi-user.target.wants/jenkins.service - /usr/lib/systemd/system/jenkins.service.
Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: disabled)
Drop-In: (etc/systemd/system/jenkins.service.d
         -override.conf
Active: active (running) since Mon 2026-09-28 10:22:54 UTC; 39ms ago
Main PID: 27472 (java)
Tasks: 40 (limit: 1055)
Memory: 27.6M
CGroup: /system.slice/jenkins.service
         └─27472 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins/war --httpPort=8081

Sep 28 10:22:50 ip-172-31-12-117.ec2.internal jenkins[27472]: [L.F]: 7b4cd5472a48496a05b2b70718f3a87
Sep 28 10:22:50 ip-172-31-12-117.ec2.internal jenkins[27472]: [L.F]:
Sep 28 10:22:50 ip-172-31-12-117.ec2.internal jenkins[27472]: [L.F]: This may also be found at: /var/lib/jenkins/secrets/initialAdminPassword
Sep 28 10:22:50 ip-172-31-12-117.ec2.internal jenkins[27472]: [L.F]:
Sep 28 10:22:50 ip-172-31-12-117.ec2.internal jenkins[27472]: [L.F]:
Sep 28 10:22:50 ip-172-31-12-117.ec2.internal jenkins[27472]: [L.F]:
Sep 28 10:22:50 ip-172-31-12-117.ec2.internal jenkins[27472]: [L.F]:
Sep 28 10:22:54 ip-172-31-12-117.ec2.internal jenkins[27472]: 2026-09-28 10:22:54.556+0000 [dd:22] INFO jenkins.InitReactorRunner$1@onAttained: Completed initialization
Sep 28 10:22:54 ip-172-31-12-117.ec2.internal jenkins[27472]: 2026-09-28 10:22:54.587+0000 [dd:24] INFO hudson.lifecycle.Lifecycle@onReady: Jenkins is fully up and running
Sep 28 10:22:54 ip-172-31-12-117.ec2.internal systemd[1]: Started jenkins.service - Jenkins Continuous Integration Server.
[root@ip-172-31-12-117 ~]# ss -tulpn | grep 8081
tcp        0      0 0.0.0.0:8081               0.0.0.0:*                  LISTEN*
tcp        0      0 0.0.0.0:8081               0.0.0.0:*                  LISTEN*
tcp6       0      0 :::8081                   :::*                        LISTEN*
tcp6       0      0 :::8081                   :::*                        LISTEN*
[root@ip-172-31-12-117 ~]# cat /var/lib/jenkins/secrets/initialAdminPassword
7b4cd5472a48496a05b2b70718f3a87
[root@ip-172-31-12-117 ~]#
```



2. Secure Jenkins server

Task 2 — Secure Jenkins Server

Objective

Secure the Jenkins server by enabling proper authentication and authorization so that only approved users can access Jenkins and permissions can be controlled using roles.

1. Open Jenkins Dashboard

Open Jenkins in the browser:

`http://<EC2-PUBLIC-IP>:8081`

Log in using the Jenkins administrator account.

2. Open Manage Jenkins

From the Jenkins dashboard:

Manage Jenkins

Then go to:

Plugins

3. Install Role-Based Authorization Strategy Plugin

Open:

Available plugins

Search for:

Role-based Authorization Strategy

Select the plugin and install it.

If Jenkins asks for restart, restart Jenkins after installation.

After login again, go to:

Manage Jenkins

→ Security

4. Configure Security Realm

Under:

Security Realm

Select:

Jenkins' own user database

This means Jenkins itself manages the user accounts.

Make sure:

Allow users to sign up

is **unchecked**.

This prevents unknown users from creating their own Jenkins accounts.

5. Configure Authorization

Under:

Authorization

change from:

Logged-in users can do anything

to:

Role-Based Strategy

Keep:

Allow anonymous read access

unchecked.

This prevents anonymous users from viewing Jenkins jobs and configuration.

6. Disable Inbound Agent Port

Under:

Agents

For:

TCP port for inbound agents

select:

Disable

because we are not using inbound Jenkins agents at this stage.

7. Keep CSRF Protection Enabled

Under:

CSRF Protection

keep:

Default Crumb Issuer

Do not disable CSRF protection.

8. Keep Legacy API Token Options Disabled

Under:

API Token

leave options such as:

Generate a legacy API token for each newly created user

and:

Allow users to manually create a legacy API token

unchecked.

9. Save Security Configuration

Click:

Save

There is no need to click **Apply** if you are using **Save**.

Remember:

Apply = save changes and stay on the same page

Save = save changes and leave the page

10. Open Role Management

After enabling Role-Based Strategy, go to:

Manage Jenkins

→ Role Management

This page contains:

Global roles

Item roles

Agent roles

11. Verify Admin Global Role

Under:

Global roles

make sure an admin role exists:

admin

and assign the admin role to your current Jenkins administrator user:

chandrashekar

The admin role should have:

Overall → Administer

This gives the administrator full Jenkins access.

Very important:

Do not remove the admin role from your current administrator account.

Otherwise, you can lock yourself out of Jenkins.

12. Leave Item Roles Empty for Now

Under:

Item roles

do not configure anything yet.

Item roles are used later to control access to specific jobs or folders.

We will use this in Task 3 when configuring limited access for:

DevOps

Testing

13. Leave Agent Roles Empty for Now

Under:

Agent roles

do not configure anything yet.

Agent roles are used to control permissions on Jenkins nodes/agents.

We will use node-related configuration in Task 4.

14. Save Role Configuration

Click:

Save

Again, clicking **Save** is enough.

AWS Security Group Check

The Jenkins server should also be secured at AWS network level.

For the EC2 Security Group:

SSH 22

Source: My IP

Custom TCP 8081

Source: My IP

Avoid:

0.0.0.0/0

for Jenkins practice unless specifically required.

This ensures only your current public IP can access Jenkins and SSH.

Task 2 Final Configuration

Security Realm

→ Jenkins' own user database 

Allow users to sign up

→ Disabled 

Authorization

→ Role-Based Strategy 

Anonymous access

→ Disabled 

Inbound agent port

→ Disabled 

CSRF Protection

→ Enabled 

Current admin user

→ Global admin role assigned 

Item roles

→ Not configured yet 

Agent roles

→ Not configured yet 

AWS 8081 access

→ Restricted to My IP 

Simple Security Architecture

User

|

| Login

v

Jenkins Authentication

Jenkins own user database

|

v

Authorization

Role-Based Strategy

|

+---- Admin

|

+---- DevOps ← Task 3

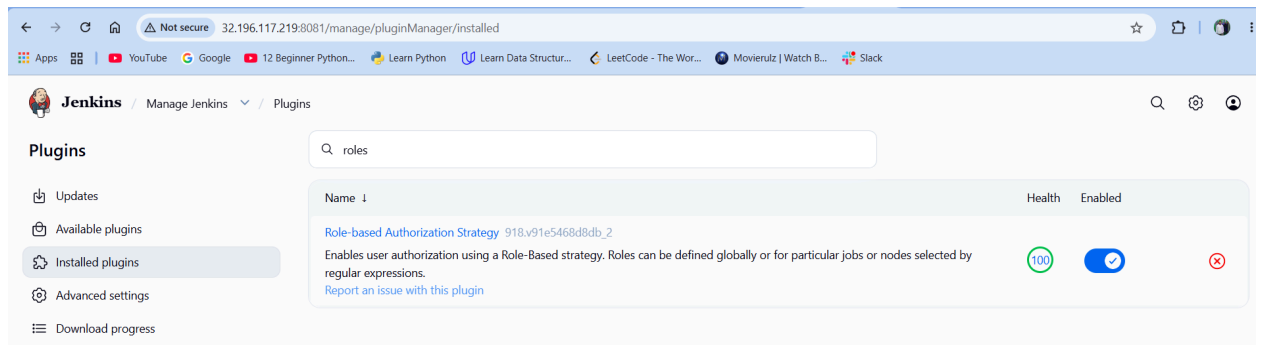
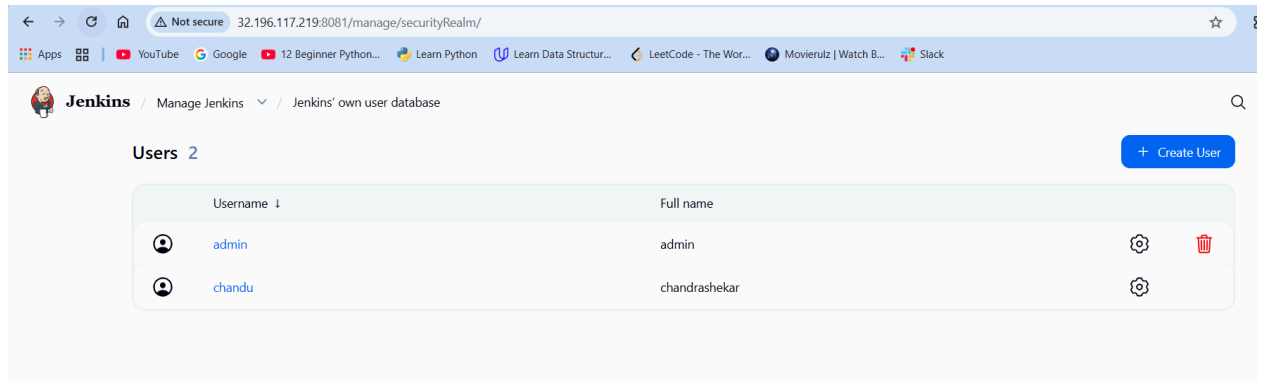
|

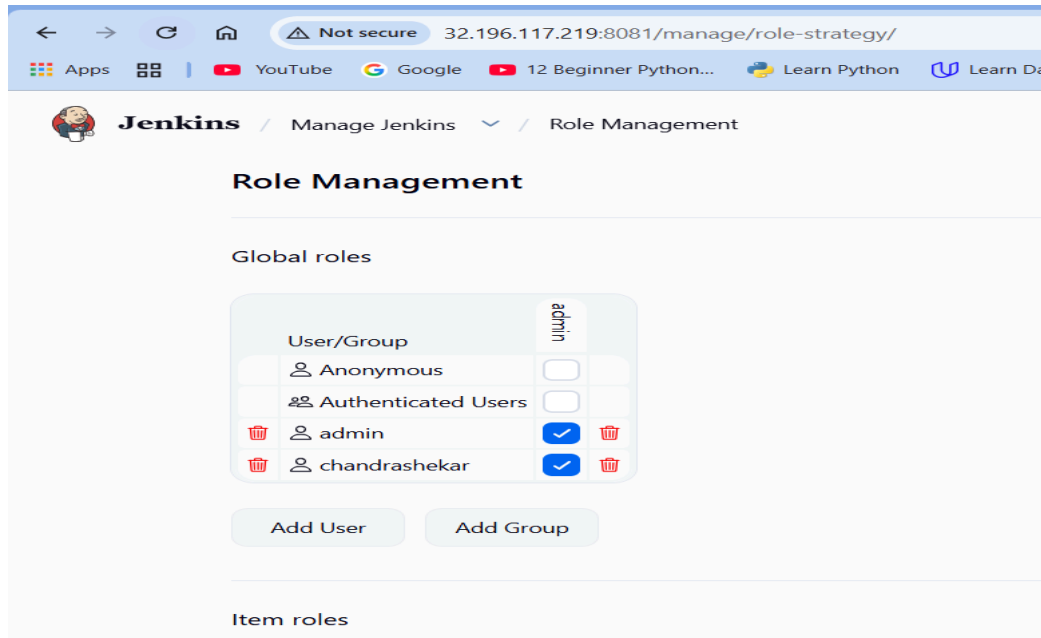
+---- Testing ← Task 3

Interview Explanation

If the interviewer asks “**How did you secure Jenkins?**”, you can say:

I enabled Jenkins' own user database for authentication and disabled self-signup. For authorization, I configured Role-Based Strategy so permissions can be assigned based on user roles. I kept anonymous access disabled, disabled the inbound agent port when it was not required, and kept CSRF protection enabled. I also restricted Jenkins port 8081 and SSH port 22 at the AWS Security Group level to my IP address.





Task 2 Result

Task 2 — Secure Jenkins Server completed successfully.

3. Create users called DevOps, Testing in Jenkins with Limited access.

Task 3 — Create DevOps and Testing Users with Limited Access

Objective

Create two Jenkins users named:

devops

testing

and provide them with limited permissions using the **Role-Based Authorization Strategy** plugin.

The administrator account should continue to have full access.

Step 1 — Create Jenkins Users

Go to:

Manage Jenkins
→ Users

Click:

Create User

Create the first user:

Username: devops
Password: *****
Full Name: DevOps
Email: test/devops email

Then create the second user:

Username: testing
Password: *****
Full Name: Testing
Email: test/testing email

Screenshot 1 to add

Take a screenshot of:

Manage Jenkins → Users

showing:

chandrashekar
devops
testing

This proves that the users were created.

Step 2 — Open Role Management

Go to:

Manage Jenkins
→ Role Management
→ Manage Roles

Create three Global Roles:

admin
devops
testing

Step 3 — Configure Admin Role

For:

admin

give:

Overall → Administer

This provides full Jenkins administrator permissions.

Step 4 — Configure DevOps Role

For:

devops

configure:

Overall

✓ Read

Job

✓ Build

✓ Cancel

✓ Read

✓ Workspace

View

✓ Read

Do not provide:

Overall/Administer

Job/Create

Job/Delete

Job/Configure

Credentials/Manage

Step 5 — Configure Testing Role

For:

testing

configure:

Overall

✓ Read

Job

✓ Build

✓ Read

✓ Workspace

View

✓ Read

Testing does not have:

Job/Cancel

Overall/Administer

Job/Create

Job/Delete

Job/Configure

Screenshot 2 to add — Very Important

Use the screenshot you already captured showing:

Global roles

admin

Overall/Administer

devops

Job/Build

Job/Cancel

Job/Read

Job/Workspace

Overall/Read

View/Read

testing

Job/Build

Job/Read

Job/Workspace

Overall/Read

View/Read

This is one of the **best screenshots for Task 3**, because it proves exactly what permissions you configured.

Add caption:

Figure 2: Global roles and limited permissions configured for DevOps and Testing users.

Step 6 — Assign Roles to Users

Return to:

Manage Jenkins
→ Role Management

Under **Global roles**, assign:

User	admin	devops	testing
chandrashekar	✓		
devops		✓	
testing			✓

In your completed setup:

devops → devops role
testing → testing role
chandrashekar → admin role
admin → admin role

Keep:

Anonymous
Authenticated Users

without these permissions.

Screenshot 3 to add — Most Important

Use your final screenshot showing:

	admin	devops	testing
devops		✓	
admin	✓		
chandrashekar	✓		
testing			✓

This proves that each user was assigned the correct role.

Caption:

Figure 3: Jenkins users assigned to their respective Global Roles.

Step 7 — Save Configuration

Click:

Save

There is no need to click both **Apply** and **Save**.

Remember:

Apply → Saves and stays on same page

Save → Saves and exits the page

Step 8 — Verify DevOps User

Open an **Incognito/Private** browser window.

Login with:

Username: devops

Password: *****

Verify that DevOps can:

View jobs

Run builds

Cancel builds

View workspace

but cannot:

Manage Jenkins

Modify Security

Delete jobs

Administer Jenkins

Screenshot 4 — Recommended

Take one screenshot after logging in as **devops**.

The screenshot should show that the user can access Jenkins/jobs but does not have administrator options.

Caption:

Figure 4: DevOps user logged in with limited Jenkins permissions.

Step 9 — Verify Testing User

Logout and login as:

Username: testing

Password: *****

Verify that Testing can:

View jobs

Run builds

View workspace

but cannot:

- Cancel builds
- Configure jobs
- Manage Jenkins
- Administer Jenkins

Screenshot 5 — Optional but useful

Take one screenshot while logged in as `testing`.

Caption:

Figure 5: Testing user logged in with restricted Jenkins permissions.

Final Task 3 Result

User	Role	Access
chandrashekar	admin	Full Administrator
devops	devops	Read/Build/Cancel/Workspace
testing	testing	Read/Build/Workspace

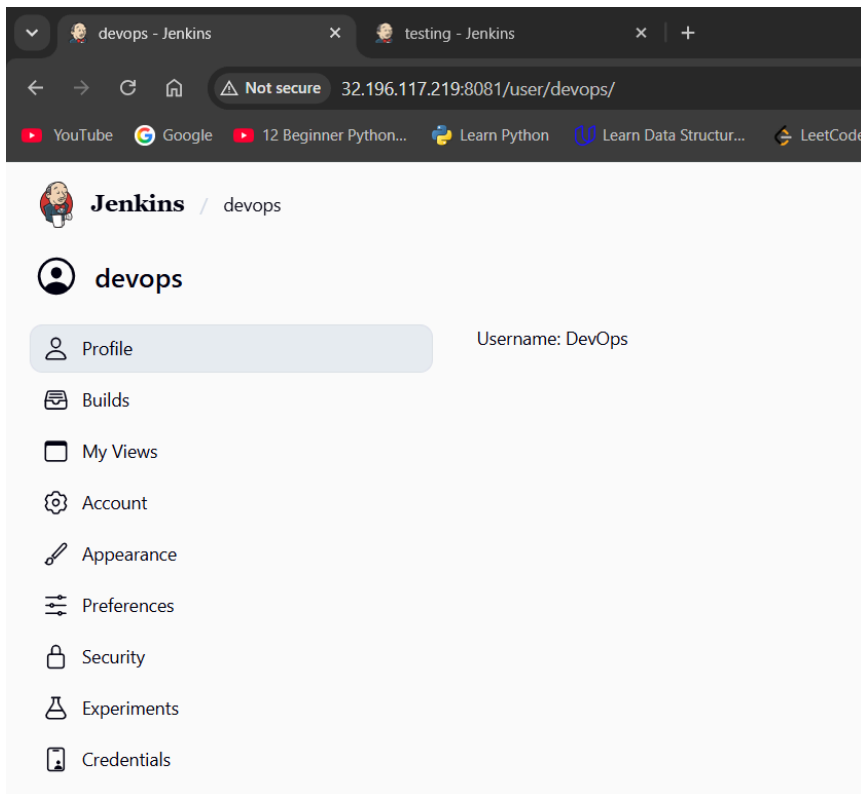
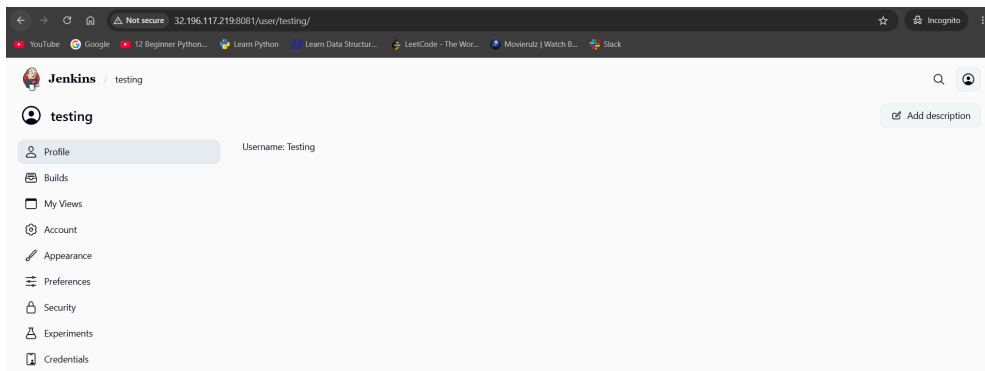
Task completed:

DevOps user created	✓
Testing user created	✓
Admin role configured	✓
DevOps limited role configured	✓
Testing limited role configured	✓
Roles assigned correctly	✓
Anonymous permissions not assigned	✓

Minimum screenshots you should include

You do **not** need screenshots for every click. I recommend **4 screenshots total**:

1. **Users page** — showing **chandrashekar**, **devops**, **testing**.
2. **Manage Roles page** — your screenshot showing **admin**, **devops**, **testing** and their permissions.
3. **Role assignment page** — your final screenshot showing **devops** → **devops**, **testing** → **testing**, **chandrashekar** → **admin**.
4. **Verification login** — preferably **devops** logged in with limited access.



4. Configure labels and restrict the jobs to execute based on label only.

5. Built-In Node

- showing label `linux`
- node online
- executor available

6. Job Configure page

- `Restrict where this project can be run` checked
- `Label Expression = linux`

7. Build History / Console Output

- showing successful build
- `stable` or `Finished: SUCCESS`

Your final task flow is:

Built-In Node

Label = linux



Label-Test-Job

Restrict where this project can be run

Label Expression = linux



Jenkins scheduler matches the label



Job executes on linux-labelled node



Build SUCCESS

So Task 4 status:

Node label configured 

Job restricted by label 

Node online 

Executor available 

Build executed successfully 

Jenkins / All / New Item

New Item

Enter an item name

Label-Test-Job

Select an item type

Pipeline

Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.

Freestyle project

Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.

Multi-configuration project

Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

Folder

Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

OK

Jenkins / All / Build History

+ New Item

Build History

Build Queue

No builds in the queue.

Build Executor Status

(0 of 1 executor busy)

Build History of Jenkins

S	Build	Time Since	1	Status	
✓	Label-Test-Job #4	1 min 34 sec		stable	
✓	Label-Test-Job #3	1 min 50 sec		stable	
✓	Label-Test-Job #2	2 min 13 sec		stable	
✓	Label-Test-Job #1	3 min 6 sec		stable	

Icon: S M L

Jenkins / Label-Test-Job / #4

Status

<> Changes

Console Output

Delete build '#4'

Edit build information

< Previous Build

Console

Download

Copy

View as plain text

started by user chandrashekar
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/Label-Test-Job
[Label-Test-Job] \$ /bin/sh -xe /tmp/jenkins1211253069187366347.sh
+ echo 'Job is running on Linux labelled node'
Job is running on Linux labelled node
+ hostname
ip-172-31-12-117.ec2.internal
+ whoami
jenkins
+ java -version
openjdk version "21.0.12.1" 2026-08-18 LTS
OpenJDK Runtime Environment Corretto-21.0.12.9.1 (build 21.0.12.1+9-LTS)
OpenJDK 64-Bit Server VM Corretto-21.0.12.9.1 (build 21.0.12.1+9-LTS, mixed mode, sharing)
Finished: SUCCESS

5.Create Three sample jobs using the below URL.

○ **Source Code:**

https://github.com/betawins/Techie_horizon_Login_app.git

Task 5 — Create Three Sample Jenkins Jobs Using GitHub Repository

Objective

Create three Jenkins Freestyle jobs using this public GitHub repository:

https://github.com/betawins/Techie_horizon_Login_app.git

The three jobs are:

1. LoginApp-Checkout
2. LoginApp-Build
3. LoginApp-Package

All jobs are restricted to the Jenkins node with label:

linux

Prerequisites

Before creating the jobs, verify Git and Maven are installed on the Jenkins server:

git --version

mvn -version

If Maven is not installed:

```
sudo dnf install maven -y
```

If Git is not installed:

```
sudo dnf install git -y
```

Job 1 — LoginApp-Checkout

Step 1 — Create the Job

Go to:

Jenkins Dashboard

→ New Item

Enter:

Job Name: LoginApp-Checkout

Select:

Freestyle project

Click:

OK

Step 2 — Restrict Job to Linux Label

Under:

General

select:

Restrict where this project can be run

Enter:

Label Expression: linux

This ensures that the job runs only on the node having the `linux` label.

Step 3 — Configure Git Repository

Under:

Source Code Management

→ Git

Enter:

Repository URL:

https://github.com/betawins/Techie_horizon_Login_app.git

Because this is a public repository:

Credentials: - none -

Under:

Branches to build

enter:

*/master

Step 4 — Add Build Step

Go to:

Build Steps

→ Add build step

→ Execute shell

Enter:

echo "Git checkout completed successfully"

pwd

ls -la

git branch

git log -1 --oneline

Click:

Save

Then:

Build Now

Result

The console output showed Jenkins successfully:

Cloning the remote Git repository

Fetching upstream changes

Checking out Revision

refs/remotes/origin/master

and completed successfully.

Screenshot 1 to add

Take the screenshot showing:

Cloning repository

Checking out Revision

Git checkout completed successfully

Finished: SUCCESS

Caption:

Figure 1: Jenkins successfully cloned and checked out the public GitHub repository.

Job 2 — LoginApp-Build

Step 1 — Create New Job

Go to:

Dashboard

→ New Item

Enter:

LoginApp-Build

Choose:

Freestyle project

Click:

OK

Step 2 — Configure Node Label

Under:

General

select:

Restrict where this project can be run

Enter:

linux

Step 3 — Configure Source Code Management

Select:

Git

Repository:

https://github.com/betawins/Techie_horizon_Login_app.git

Credentials:

None

Branch:

*/master

Step 4 — Configure Maven Build

Under:

Build Steps

→ Add build step

→ Execute shell

enter:

```
echo "Starting Maven build"
```

```
mvn clean compile
```

```
echo "Maven build completed"
```

Click:

Save

→ Build Now

Result

The console output showed:

```
[INFO] Compiling 11 source files
```

```
[INFO] BUILD SUCCESS
```

```
Maven build completed
```

```
Finished: SUCCESS
```

This confirms that the Java source code compiled successfully.

Screenshot 2 to add

Use the screenshot that shows:

```
BUILD SUCCESS
```

```
Maven build completed
```

```
Finished: SUCCESS
```

Caption:

Figure 2: Maven compile job completed successfully in Jenkins.

Job 3 — LoginApp-Package

Step 1 — Create New Job

Go to:

Dashboard

→ New Item

Enter:

LoginApp-Package

Choose:

Freestyle project

Click:

OK

Step 2 — Configure Linux Label

Under:

General

select:

Restrict where this project can be run

Enter:

linux

Step 3 — Configure Git Repository

Under:

Source Code Management

→ Git

enter:

Repository URL:

https://github.com/betawins/Techie_horizon_Login_app.git

Credentials:

None

Branch:

*/master

Step 4 — Initial Package Command

Initially, we used:

```
echo "Starting Maven package"
```

```
mvn clean package
```

```
echo "Package created successfully"
```

```
ls -lh target/
```

The build failed.

Issue Faced

The console showed:

Failed to execute goal

org.apache.maven.plugins:maven-war-plugin:2.2:war

and:

Cannot access defaults field of Properties

The build ended with:

Finished: FAILURE

The issue was caused by the project using an old:

maven-war-plugin:2.2

which was not compatible with the current Java 21 environment.

Troubleshooting Step

To confirm the error, we ran Maven manually from the Jenkins workspace.

The Jenkins workspace was:

```
cd /var/lib/jenkins/workspace/LoginApp-Package
```

Then tested as the Jenkins user:

```
sudo -u jenkins -H bash -c \
```

```
'cd /var/lib/jenkins/workspace/LoginApp-Package && mvn clean package -DskipTests'
```

The same `maven-war-plugin:2.2` error occurred.

This confirmed that the issue was with the Maven plugin and not Jenkins itself.

Step 5 — Fix the Maven WAR Plugin

We changed the Jenkins Execute Shell commands to:

```
echo "Starting application packaging"
```

```
mvn clean compile
```

```
mvn org.apache.maven.plugins:maven-war-plugin:3.5.1:war -DskipTests
```

```
echo "Package created successfully"
```

```
ls -lh target/
```

Then clicked:

Save

→ Build Now

Final Result

The console showed:

Building war:

/var/lib/jenkins/workspace/LoginApp-Package/target/vprofile-v2.war

Then:

[INFO] BUILD SUCCESS

Package created successfully

The generated artifact was:

vprofile-v2.war

Size:

17M

Finally:

Finished: SUCCESS

Screenshot 3 to add

Use the screenshot showing:

Building war:

.../target/vprofile-v2.war

BUILD SUCCESS

Package created successfully

vprofile-v2.war

Finished: SUCCESS

Caption:

Figure 3: Jenkins successfully packaged the application and generated the WAR artifact.

Task 5 Workflow

GitHub Repository



Jenkins Checkout Job



Clone source code



LoginApp-Build



mvn clean compile



BUILD SUCCESS



LoginApp-Package



Compile application



WAR packaging

↓
vprofile-v2.war
↓
BUILD SUCCESS

Final Task 5 Status

LoginApp-Checkout  SUCCESS

LoginApp-Build  SUCCESS

LoginApp-Package  SUCCESS

Generated artifact:

target/vprofile-v2.war

Troubleshooting Learned

The important real-time issue we faced was:

maven-war-plugin:2.2

↓
Incompatible with Java 21

↓
Package build failed



Used maven-war-plugin:3.5.1



WAR file created successfully

Task 5 — Completed successfully.

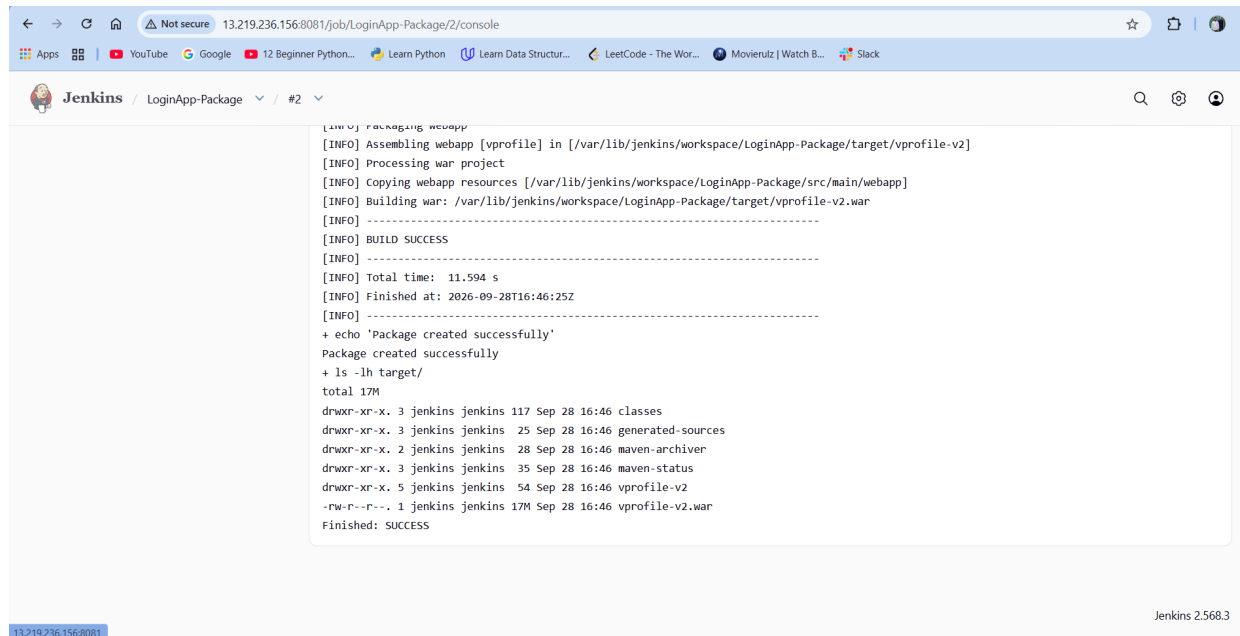
The screenshot shows the Jenkins console output for a build named 'LoginApp-Checkout'. The left sidebar contains links to 'Status', 'Changes', 'Console Output', 'Delete build #1', 'Git Build Data', and 'Edit build information'. The main console area displays the following text:

```
Started by user chandrashekar
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/LoginApp-Checkout
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/betawins/Techie_horizon_login_app.git
> git init /var/lib/jenkins/workspace/LoginApp-Checkout # timeout=10
Fetching upstream changes from https://github.com/betawins/Techie_horizon_login_app.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/betawins/Techie_horizon_login_app.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git config remote.origin.url https://github.com/betawins/Techie_horizon_login_app.git # timeout=10
> git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
> git rev-parse refs/remotes/origin/master^{commit} # timeout=10
Checking out Revision ef9c453d4993843ac49d37321316c2588f154495 (refs/remotes/origin/master)
> git config core.sparsecheckout # timeout=10
> git checkout -f ef9c453d4993843ac49d37321316c2588f154495 # timeout=10
Commit message: "sabit code for testing"
First time build. Skipping changelog.
[LoginApp-Checkout] $ /bin/sh -xe /tmp/jenkins34519975657160851.sh
+ echo 'Git checkout completed successfully'
```

The screenshot shows the Jenkins console output for a build named 'LoginApp-Build'. The left sidebar contains links to 'Status', 'Changes', 'Console Output', 'Delete build #1', 'Git Build Data', and 'Edit build information'. The main console area displays the following text:

```
Progress (3): 640 kB | 121 kB | 40/45 kB
Progress (3): 640 kB | 121 kB | 45/45 kB
Progress (3): 640 kB | 121 kB | 45 kB

Downloaded from central: https://repo.maven.apache.org/maven2/junit/junit/3.8.2/junit-3.8.2.jar (121 kB at 410 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/com/google/collections/google-collections/1.0/google-collections-1.0.jar (640 kB at 2.2 MB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/commons-logging/commons-logging-api/1.1/commons-logging-api-1.1.jar (45 kB at 140 kB/s)
[INFO] Changes detected - recompiling the module!
[WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
[INFO] Compiling 11 source files to /var/lib/jenkins/workspace/LoginApp-Build/target/classes
[INFO] BUILD SUCCESS
[INFO] Total time: 22.016 s
[INFO] Finished at: 2026-09-28T16:12:27Z
[INFO] + echo 'Maven build completed'
Maven build completed
Finished: SUCCESS
```



```
[INFO] Packaging webapp
[INFO] Assembling webapp [vprofile] in [/var/lib/jenkins/workspace/LoginApp-Package/target/vprofile-v2]
[INFO] Processing war project
[INFO] Copying webapp resources [/var/lib/jenkins/workspace/LoginApp-Package/src/main/webapp]
[INFO] Building war: /var/lib/jenkins/workspace/LoginApp-Package/target/vprofile-v2.war
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 11.594 s
[INFO] Finished at: 2026-09-28T16:46:25Z
[INFO] -----
+ echo 'Package created successfully'
Package created successfully
+ ls -lh target/
total 17M
drwxr-xr-x. 3 jenkins jenkins 117 Sep 28 16:46 classes
drwxr-xr-x. 3 jenkins jenkins 25 Sep 28 16:46 generated-sources
drwxr-xr-x. 2 jenkins jenkins 28 Sep 28 16:46 maven-archiver
drwxr-xr-x. 3 jenkins jenkins 35 Sep 28 16:46 maven-status
drwxr-xr-x. 5 jenkins jenkins 54 Sep 28 16:46 vprofile-v2
-rw-r--r--. 1 jenkins jenkins 17M Sep 28 16:46 vprofile-v2.war
Finished: SUCCESS
```

6. Create one Jenkins job using GitHub Private repository.

Task 6 — Create Jenkins Job Using GitHub Private Repository

Objective

Create a Jenkins Freestyle job that clones source code from a **private GitHub repository** using Jenkins Credentials.

Private repository used:

<https://github.com/chandrashekar0532/jenkins-private-demo.git>

Jenkins credential ID used:

github-private-repo

1. Create Private GitHub Repository

Go to GitHub:

GitHub

→ Repositories

→ New

Create:

Repository name: jenkins-private-demo

Visibility: Private

Optional:

Add README file

Then click:

Create repository

Repository created:

<https://github.com/chandrashekar0532/jenkins-private-demo.git>

Screenshot 1 to add

Take a screenshot showing:

jenkins-private-demo

Private

Caption:

Figure 1: Private GitHub repository created for Jenkins integration.

2. Create GitHub Personal Access Token

In GitHub go to:

Profile

→ Settings

→ Developer settings

→ Personal access tokens

→ Fine-grained tokens

→ Generate new token

Configure:

Token name: jenkins-private-repo
Resource owner: chandrashekar0532
Repository access: Only select repositories
Repository: jenkins-private-demo

Under Repository permissions:

Contents → Read-only

Then click:

Generate token

Copy the generated token temporarily.

Important:

Do not expose the token in screenshots.
Do not put the token directly in the Jenkins job or repository URL.

3. Open Jenkins Credentials

In Jenkins:

Manage Jenkins
→ Credentials
→ System
→ Global credentials

Click:

Add Credentials

4. Add GitHub Credentials

Select:

Kind: Username with password

Enter:

Scope: Global

Username:

chandrashekar0532

Password:

<GitHub Personal Access Token>

ID:

github-private-repo

Description:

GitHub private repository access

Click:

Create

Screenshot 2 to add

Take a screenshot of the Jenkins Credentials page showing:

github-private-repo

chandrashekar0532/*****

System - Global

GitHub private repository access

Do not show the actual PAT.

Caption:

Figure 2: GitHub Personal Access Token stored securely in Jenkins Credentials.

5. Create Jenkins Freestyle Job

Go to:

Jenkins Dashboard

→ New Item

Enter:

Private-Repo-Checkout

Select:

Freestyle project

Click:

OK

6. Restrict Job to Linux Node

Under:

General

enable:

Restrict where this project can be run

Enter:

Label Expression: linux

This ensures the job runs only on the Jenkins node having the `linux` label.

7. Configure Private GitHub Repository

Under:

Source Code Management

→ Git

Enter:

Repository URL:

<https://github.com/chandrashekar0532/jenkins-private-demo.git>

Under Credentials select:

github-private-repo

or the displayed credential:

chandrashekar0532/***** (github-private-repo)

8. Configure Branch

Under:

Branches to build

enter:

*/main

because the private repository uses the `main` branch.

9. Add Build Step

Go to:

Build Steps

→ Add build step

→ Execute shell

Enter:

echo "Private GitHub repository checkout successful"

pwd

ls -la

git branch

git log -1 --oneline

Then click:

Save

10. Run Jenkins Job

Click:

Build Now

Then open:

Build History

→ #1

→ Console Output

11. Console Output Verification

Our build showed:

Started by user chandrashekar

Running as SYSTEM

Building in workspace

/var/lib/jenkins/workspace/Private-Repo-Checkout

Jenkins used the stored credential:

using credential github-private-repo

It successfully cloned:

<https://github.com/chandrashekar0532/jenkins-private-demo.git>

Jenkins fetched the repository and checked out:

refs/remotes/origin/main

The commit was:

559ac99 Initial commit

The shell step displayed:

Private GitHub repository checkout successful

Workspace contents included:

.git
README.md

Finally:

Finished: SUCCESS

12. Final Result

The private GitHub repository was successfully accessed by Jenkins using stored credentials.

Private Repository



GitHub Fine-Grained PAT



Jenkins Credentials

github-private-repo



Private-Repo-Checkout Job



Git Clone



main branch checkout



Finished: SUCCESS

Task 6 Status

Private GitHub repository created 

Fine-grained PAT created 

Repository read permission given 

PAT stored in Jenkins Credentials 

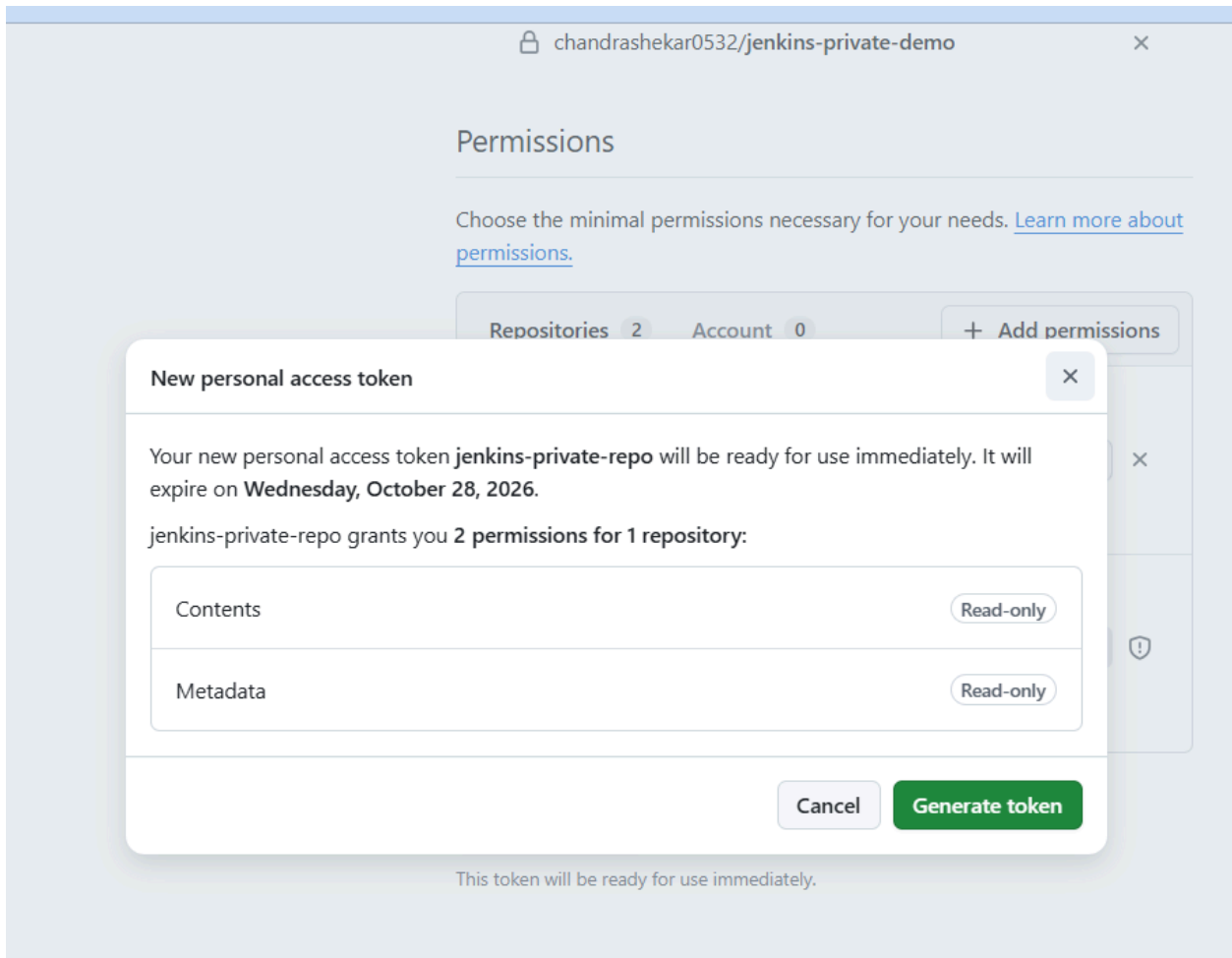
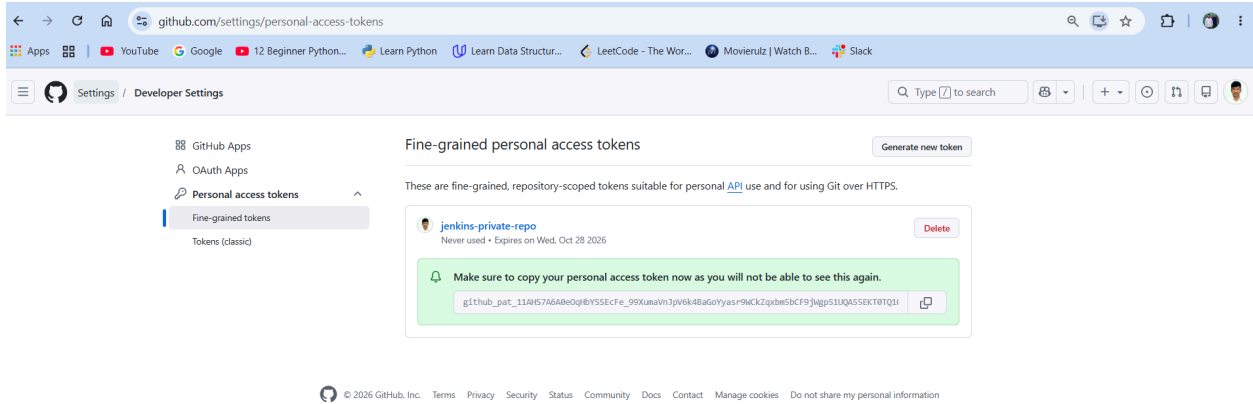
- Private repository job created 
- Credential selected in Git SCM 
- main branch configured 
- Private repository cloned 
- Build completed successfully 

Screenshots to include

I recommend **4 screenshots**:

1. **Private GitHub repository**
 - `jenkins-private-demo`
 - `Private`
2. **Jenkins Credentials**
 - `github-private-repo`
 - username visible
 - password/token masked
3. **Job configuration**
 - private repository URL
 - selected Jenkins credential
 - branch `*/main`
 - label `linux`
4. **Console Output**
 - `using credential github-private-repo`
 - `Checking out Revision`
 - `Private GitHub repository checkout successful`
 - `README.md`
 - `Finished: SUCCESS`

The **Console Output screenshot is the most important proof** because it demonstrates that Jenkins successfully authenticated to a private repository and cloned it.



The screenshot shows the Jenkins web interface. The browser address bar indicates the URL is `13.219.236.156:8081/job/Private-Repo-Checkout/1/console`. The Jenkins page header shows the job name 'Private-Repo-Checkout' and build number '#1'. On the left sidebar, the 'Console Output' tab is selected. The main console area displays the following text:

```
Started by user chandrashekar
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/Private-Repo-Checkout
The recommended git tool is: NONE
using credential github-private-repo
Cloning the remote Git repository
Cloning repository https://github.com/chandrashekar0532/jenkins-private-demo.git
> git init /var/lib/jenkins/workspace/Private-Repo-Checkout # timeout=10
Fetching upstream changes from https://github.com/chandrashekar0532/jenkins-private-demo.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
using GIT_ASKPASS to set credentials GitHub private repository access
> git fetch --tags --force --progress -- https://github.com/chandrashekar0532/jenkins-private-demo.git +refs/heads/*:refs/remotes/origin/* #
timeout=10
> git config remote.origin.url https://github.com/chandrashekar0532/jenkins-private-demo.git # timeout=10
> git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 559ac9962106c2458c505f9daba02de31cfa9bd2 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 559ac9962106c2458c505f9daba02de31cfa9bd2 # timeout=10
Commit message: "Initial commit"
First time build. Skipping changelog.
[Private-Repo-Checkout] $ /bin/sh -xe /tmp/jenkins1308515019843279001.sh
```

Task 6 — Completed successfully.